EECE7352 Computer Architecture

Homework 4

Sakthivel P. Sivakumar

NUID: 002312294

Part A: (40 Points)

The Dinero cache simulator. We are using **Dinero IV**

1. Assume that you have a direct-mapped 4KB instruction cache with a 32B block size. Develop an address stream that will touch every cache line once, but no more than once.

Ans:

Given: Instruction Cache: 4KB, Block Cache: 32 B

No. of Cache lines = (Total Cache Size/ Block Size) = 128

1. Command to Generate Trace File:

   for ((a=0; a<4096; a+=32)); do printf "2 %x\n" $a; done > trace1.txt

2. Command to Run DineroIV with the trace1.txt

   dineroIV -l1-isize 4K -l1-ibsize 32 -informat d < trace1.txt

Run Output:

```
---All rights reserved.
---Copyright (C) 1985, 1989 Mark D. Hill.  All rights reserved.
---See -copyright option for details

---Summary of options (-help option gives usage information).

-l1-isize 4096
-l1-ibsize 32
-l1-isbsize 32
-l1-iassoc 1
-l1-irepl l
-l1-ifetch d
-skipcount 0
-flushcount 0
-maxcount 0
-stat-interval 0
-informat d
-on-trigger 0x0
-off-trigger 0x0

---Simulation begins.
---Simulation complete.
l1-icache
 Metrics                    Total         Instrn          Data          Read         Write          Misc
 -----------------          ------        ------        ------        ------        ------        ------
 Demand Fetches                128           128             0             0             0             0
  Fraction of total         1.0000        1.0000        0.0000        0.0000        0.0000        0.0000

 Demand Misses                 128           128             0             0             0             0
  Demand miss rate          1.0000        1.0000        0.0000        0.0000        0.0000        0.0000

 Multi-block refs                0
 Bytes From Memory            4096
 ( / Demand Fetches)         32.0000
 Bytes To Memory                 0
 ( / Demand Writes)          0.0000
 Total Bytes r/w Mem          4096
 ( / Demand Fetches)         32.0000

---Execution complete.
[sakthivelps@Theta PartA]$
```

2. Assume the same size and line size as the instruction cache in part 1, but now the instruction cache is 4-way set associative, with LRU replacement. The total cache space is still 4KB with a 32B block size, but now you have 1/4 the number of indices. Develop an address stream that touch every cache index, producing 4 misses and 5 hits per index, and only includes 4 unique addresses per cache index.

Ans:

Given that the instruction cache is 4-way set associative,

The no of Indices = (Total Cache size/ (Block size x Associativity)) = 128/4 = 32

Since it is mentioned the instruction cache is 4-way set, each set can hold 4 cache lines. Each index maps to addresses spaced by 32B x 32 sets = 0x400. And 4 unique addresses are chosen per index that differ by 0x400 each.

Eg: (index 0)

2 0

2 400

2 800

2 c00

From the above example we can see that, each of these addresses are mapping to the same set, filling the 4 ways, initially we get 4 misses and when we access them again we get hits as they are already present in the cache (4+4)

so, for 4 misses and 5 hits, the sequence is

2 0

2 400

2 800

2 c00

2 0

2 400

2 800

2 c00

2 0

1. Command to Generate Trace File:

for((s=0;s<32;s++));do b=$((s*32));for a in $(seq 0 1024 3072);do for i in {1..3};do printf "2 %x\n" $((b+a));done;done;done>trace2.txt

2. Command to Run DineroIV with the trace1.txt

dineroIV -l1-isize 4K -l1-ibsize 32 -l1-iassoc 4 -informat d < trace2.txt

Run Output:

```
---Summary of options (-help option gives usage information).

-l1-isize 4096
-l1-ibsize 32
-l1-isbsize 32
-l1-iassoc 4
-l1-irepl l
-l1-ifetch d
-skipcount 0
-flushcount 0
-maxcount 0
-stat-interval 0
-informat d
-on-trigger 0x0
-off-trigger 0x0

---Simulation begins.
---Simulation complete.
l1-icache
 Metrics                    Total          Instrn          Data           Read          Write           Misc
 -----------------          ------         ------         ------         ------         ------         ------
 Demand Fetches                384            384              0              0              0              0
  Fraction of total          1.0000         1.0000         0.0000         0.0000         0.0000         0.0000

 Demand Misses                 128            128              0              0              0              0
  Demand miss rate           0.3333         0.3333         0.0000         0.0000         0.0000         0.0000

 Multi-block refs                0
 Bytes From Memory            4096
 ( / Demand Fetches)         10.6667
 Bytes To Memory                 0
 ( / Demand Writes)          0.0000
 Total Bytes r/w Mem          4096
 ( / Demand Fetches)         10.6667

---Execution complete.
[sakthivelps@Theta PartA]$
```

3. Repeat part 2, develop an address stream of addresses, but now the cache is 2-way set associative. Produce 5 misses and 5 hits per index, again with only 3 unique address per index, but produce an interleaving pattern of Miss-Hit-Miss-Hit-Miss-Hit-…….

For 2-way set , no of indices = 128/2 = 64

1. Command to Generate Trace File:

for((s=0;s<32;s++));do b=$((s*32));for a in $(seq 0 1024 3072);do for i in {1..3};do printf "2 %x\n" $((b+a));done;done;done>trace2.txt

2. Command to Run DineroIV with the trace1.txt

dineroIV -l1-isize 4K -l1-ibsize 32 -l1-iassoc 4 -informat d < trace2.txt

Run Output:

```
---Summary of options (-help option gives usage information).

-l1-isize 4096
-l1-ibsize 32
-l1-isbsize 32
-l1-iassoc 2
-l1-irepl l
-l1-ifetch d
-skipcount 0
-flushcount 0
-maxcount 0
-stat-interval 0
-informat d
-on-trigger 0x0
-off-trigger 0x0

---Simulation begins.
---Simulation complete.
l1-icache
 Metrics                    Total          Instrn          Data           Read          Write           Misc
 -----------------          ------         ------         ------         ------         ------         ------
 Demand Fetches                640            640              0              0              0              0
  Fraction of total          1.0000         1.0000         0.0000         0.0000         0.0000         0.0000

 Demand Misses                 320            320              0              0              0              0
  Demand miss rate           0.5000         0.5000         0.0000         0.0000         0.0000         0.0000

 Multi-block refs               0
 Bytes From Memory          10240
 ( / Demand Fetches)        16.0000
 Bytes To Memory                0
 ( / Demand Writes)         0.0000
 Total Bytes r/w Mem        10240
 ( / Demand Fetches)        16.0000

---Execution complete.
[sakthivelps@Theta PartA]$ |
```

4. Assume that you have a 2-way set associate 32KB data cache with a 32B block size. Develop an address stream that will generate 5 hits and 5 misses per index. Make sure that your stream includes both loads and stores.

Given: Data Cache: 32 KB , Block Size: 32B
No of Indices = 32K/32 = 512

The difference in data cache is that it handles load (0) and store (1) operations.

Run Output:

```
---Summary of options (-help option gives usage information).

-l1-dsize 32768
-l1-dbsize 32
-l1-dsbsize 32
-l1-dassoc 2
-l1-drepl l
-l1-dfetch d
-l1-dwalloc a
-l1-dwback a
-skipcount 0
-flushcount 0
-maxcount 0
-stat-interval 0
-informat d
-on-trigger 0x0
-off-trigger 0x0

---Simulation begins.
---Simulation complete.
l1-dcache
 Metrics                    Total          Instrn          Data           Read          Write           Misc
 -----------------          ------         ------         ------         ------         ------         ------
 Demand Fetches                5120              0           5120           2560           2560              0
  Fraction of total          1.0000         0.0000         1.0000         0.5000         0.5000         0.0000

 Demand Misses                 2560              0           2560           2560              0              0
  Demand miss rate           0.5000         0.0000         0.5000         1.0000         0.0000         0.0000

 Multi-block refs               0
 Bytes From Memory          81920
 ( / Demand Fetches)        16.0000
 Bytes To Memory            81920
 ( / Demand Writes)         32.0000
 Total Bytes r/w Mem        163840
 ( / Demand Fetches)        32.0000

---Execution complete.
[sakthivelps@Theta PartA]$ |
```

5. Given a partially specified instruction cache organization, develop one or multiple instruction address reference streams that can detect the set associativity and the total size of an instruction cache. Assume that you know that the block size is 64B and that the cache size will be no larger than 32KB. Develop the stream(s) and discuss how you figured out the total size and the associativity.

Given: Block Size: 64 B; Total Cache Size is <=32 KB

The task is to develop multiple streams and discuss how I figured out the total cache size and set associativity.

The detection strategies rely on creating conflicting patterns. These are used in accessing addresses that map to the same cache set to observe when cache lines get evicted.

w.k.t, **C = S x A x B**

where, S = No. of Sets, B = Block size, A = Associativity, C = cache size

Detection Methodology

Part 1: Detect the Total Cache Size

For this we use a stride pattern to determine when we start seeing misses due to cache capacity

Test Stream: (Capacity Detection)

Access sequential blocks with stride = 64B

Touch N blocks where N increases until we exceed capacity

Pattern: Access blocks at 0, 64, 128, 192, ... then revisit them

Address Stream

First pass - touch 512 blocks (32KB worth if all fit)

2 0 # Block 0

2 40 # Block 1

2 80 # Block 2

2 c0 # Block 3

...

2 7fc0 # Block 511 (32KB / 64B = 512 blocks)

Second pass - revisit blocks to detect capacity

2 0 # Should HIT if cache ≥ 32KB, MISS if smaller

2 40

2 80

....

Analysis

- The blocks are accessed sequentially from address 0 with stride 64B
- After touching k blocks, revisit them
- Count the no. of blocks that result in HITs on revisit
- Capacity = No. of HITs x 64 (approx.)

If the no of HITs is 512 then the total cache size is 32 KB, if no of HITS is 256, cache size is 16 KB and similarly different total cache size for different HIT rates

Part 2: Detect Set Associativity

Once we have found the cache size, we can determine associativity by creating **set conflicts**

Formula:

Set Stride = No. of Sets x Block Size = (Total Cache Size/ Associativity) x 64B

Test Stream (Associativity Detection)

For easy calculation purposes I have assumed the cache size is 16KB

Test for Direct-Mapped (1-way)

If direct mapped: Set_Stride = 16KB

Access pattern: 0, 4000, 8000, 0 (should get MISS, MISS, MISS, MISS)

2 0 # Miss - load to set 0

2 4000 # Miss - conflicts with 0, evicts it (if direct-mapped)

2 0 # Miss if direct-mapped (was evicted), HIT if 2-way or higher

2 4000 # Analysis point

```
# Test for 2-way set associative
# If 2-way: Set_Stride = 8KB = 2000 (hex)
# Access pattern with 3 conflicting addresses

2 0      # Miss - load to set 0, way 0
2 2000   # Miss - load to set 0, way 1
2 4000   # Miss - load to set 0, evicts way 0 (LRU) if 2-way
2 0      # Miss if 2-way, HIT if 4-way or higher
2 2000   # HIT (should still be present)
2 4000   # HIT (should still be present)
```

Completed Detection Stream

```
#  PHASE 1: Size Detection
# Touch 512 blocks sequentially
2 0
2 40
2 80
...
2 7fc0

# Revisit first blocks - count HITs to determine size
2 0
2 40
...

#  PHASE 2: Associativity Detection
# Assuming 16KB detected, test associativities

# Test Direct-Mapped (stride = 4000)
2 0
2 4000
```

```
2 0      # Check if MISS (direct) or HIT (>1-way)


# Test 2-way (stride = 2000)
2 10000
2 12000
2 14000
2 10000   # Check if MISS (2-way) or HIT (>2-way)


# Test 4-way (stride = 1000)
2 20000
2 21000
2 22000
2 23000
2 24000
2 20000   # Check if MISS (4-way) or HIT (>4-way)
```

6. Repeat part 5, but now develop an instruction stream that will determine the replacement algorithm. Again, discuss how you determined this.

To determine the replacement policy, I constructed an instruction stream that forces multiple blocks into the same cache set. From Part 5, I know the block size (64B) and the set-index stride, so I picked five addresses A, B, C, D, and E that all map to the same index by adding multiples of the stride value. Example:

A = 0
B = Stride
C = 2·Stride
D = 3·Stride
E = 4·Stride

First, I accessed A, B, C, and D to fill all ways of the set. Then I re-accessed A and B to make them the most recently used. After this, I accessed E, which forces an eviction since the set is already full.

By probing specific blocks afterwards, I can tell which block was evicted:

- If C - the least recently used was evicted, the policy is LRU.
- If A -the oldest inserted was evicted, the policy is FIFO.
- If different blocks are evicted on repeated runs, the replacement is Random.

Thus, the eviction pattern after inserting the 5th block reveals the replacement algorithm used by the instruction cache.